```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

6. Tutorial - Inheritance & Polymorphie

Kuzey Aydınhan | 12. December 2025

Contents

1. Overview

2. Iterator

3. Inheritance

4. Dynamic Binding

5. Absstract Class

6. Command Pattern

Information

Important

No tutorial next week! See you on 09. Jan

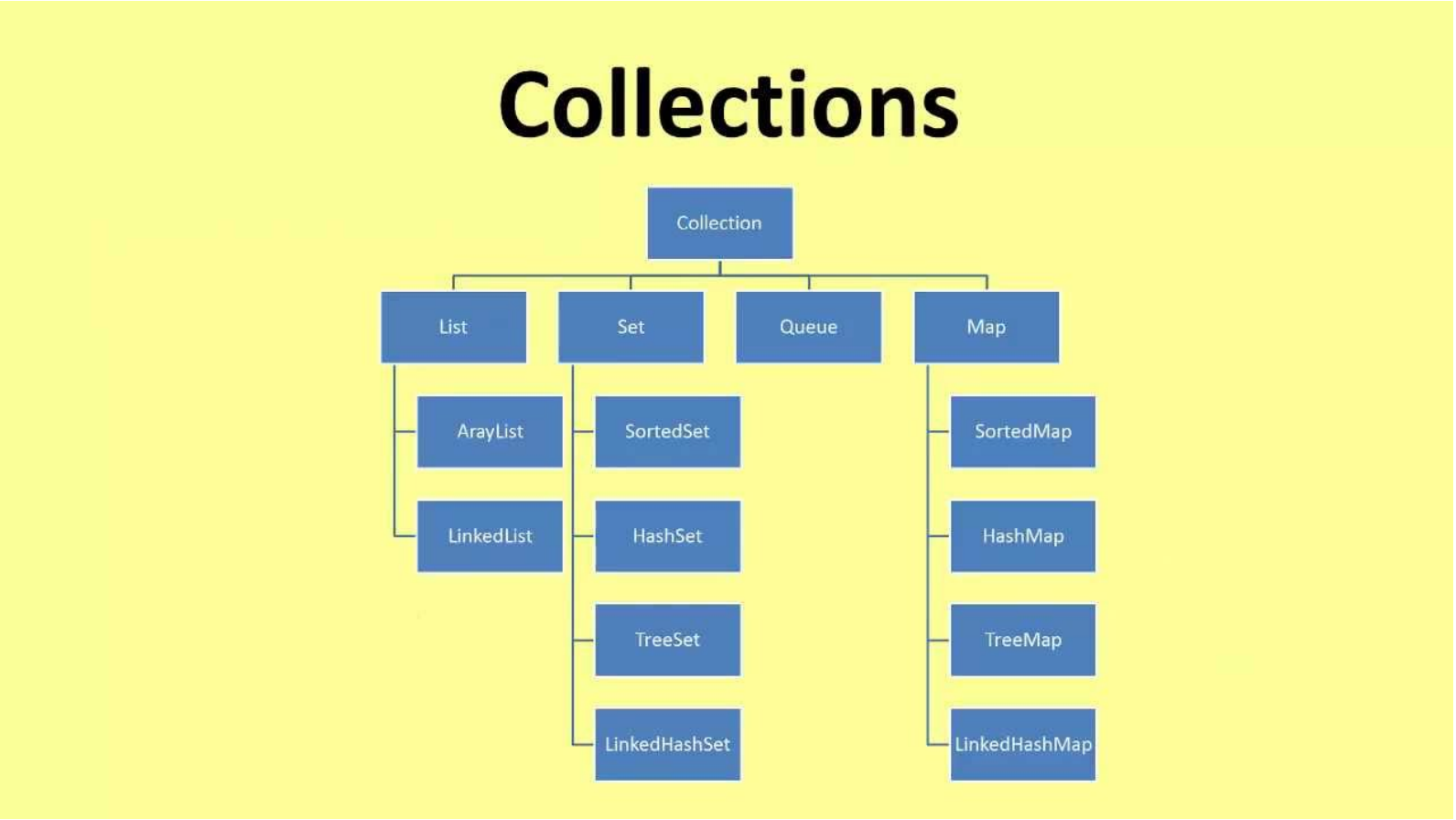
Dates

- The Präsenzübung is on 21. Jan

Last Week

Abstract Data Types

- Common structures
- Extendable
- Information hiding
- Implementation can differ



Last Week

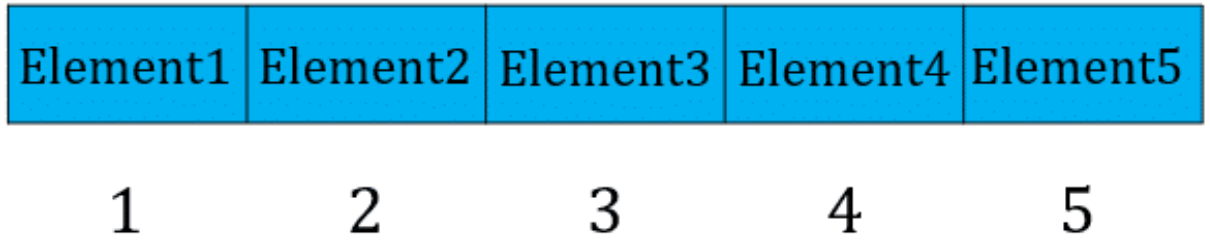
Lists

- Implement base operations:
  - `add(item)`
  - `remove(item)`
  - `size()`
- Only accept non-primitives
- Two different implementations in Java:
  - `ArrayList`
  - `LinkedList`

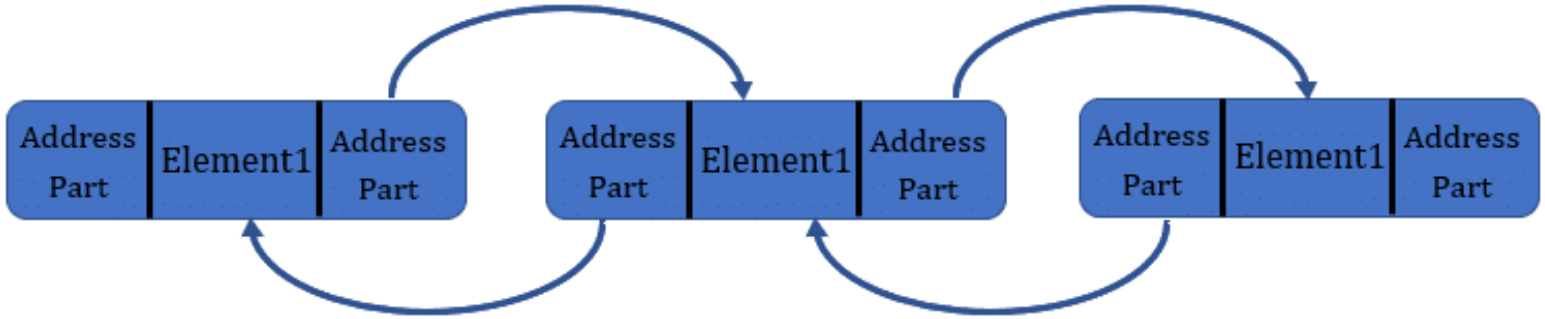
```java
public static void main(String[] args) {
    List<Player> players = new LinkedList<Player>();
    players.add(player1);
    players.add(player2);

    List<Person> people = new ArrayList<Person>();
    ..
}

```

Last Week

ArrayList



LinkedList



Homework

Utility Classes

Bad Code:

```java
public UtilityClass {
    public static int calculate() {
        // ...
    }

    public static String parse() {
        // ...
    }
}

```

If a class is only for static methods → is a utility class

- Utility classes must be **`final`**
- Utility classes must have **`private`** constructors

Correct:

```java
public final UtilityClass {
    private UtilityClass() {
        // private constructor
    }

    public static int calculate() { ... }

    public static String parse() { ... }
}

```

Motivation

- Iterator
- Inheritance
- Dynamic Binding
- Abstract Classes
- Command Pattern

Iterator

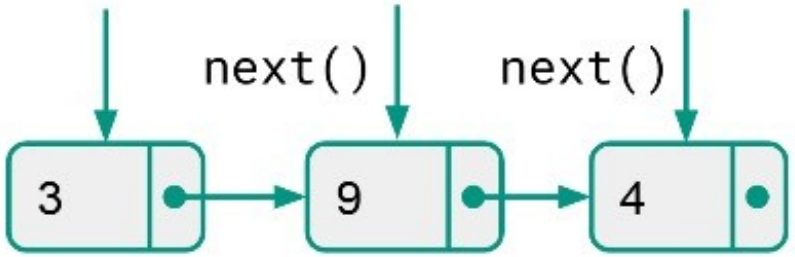
Idea

- Working with `for` loops
- Overview:
  1. Start with inital value
  2. Execute something
  3. Increase value
  4. **Break** when no element left
- **Generalized version: Iterator**
  - Usage with `for-each`
  - Must implement the interface `java.util.Iterator` (More in future)

Iterator

Implementation

- Initial value → Contstructor
- Execute something with current value
- Increase value → call `next()`
- Any element left → call `hasNext()`



Inheritance

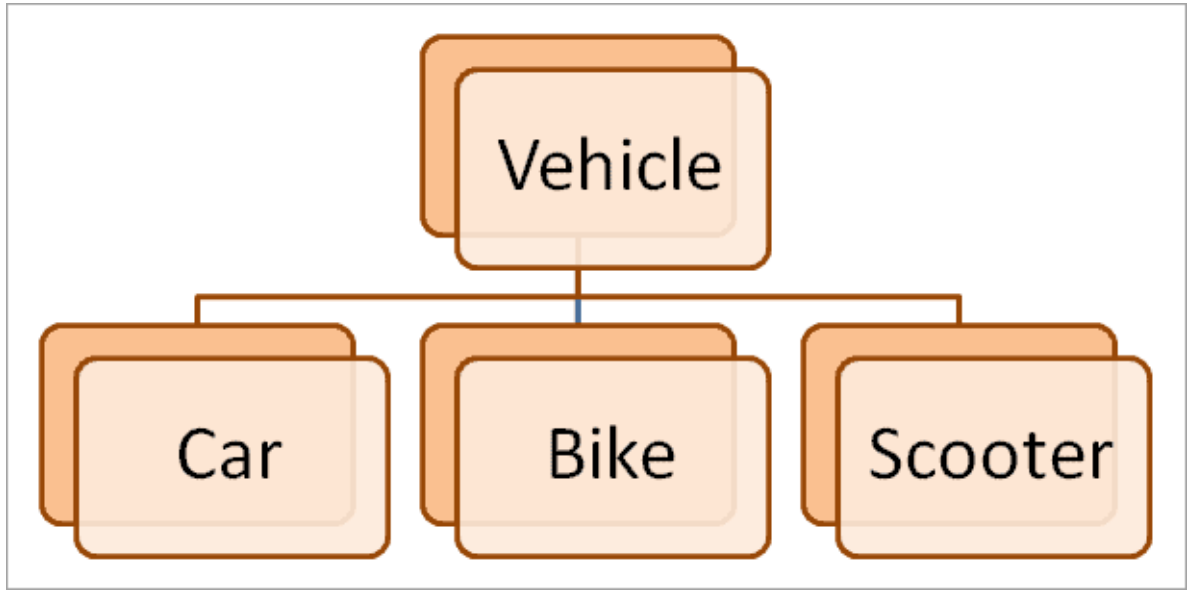
Problem

- How do we model cars?
- How do we model bikes?
- What are the similarities?
  $\rightarrow$ both are vehicles
- Problem so far: **Code duplicates**

```
class Car {
    String name;
    String brand;
    Color color;

    void turnEngineOn() {...}
}

class Bike {
    String name;
    String brand;
    Color color;

    void turnEngineOn() {...}
}

```

Inheritance

Idea



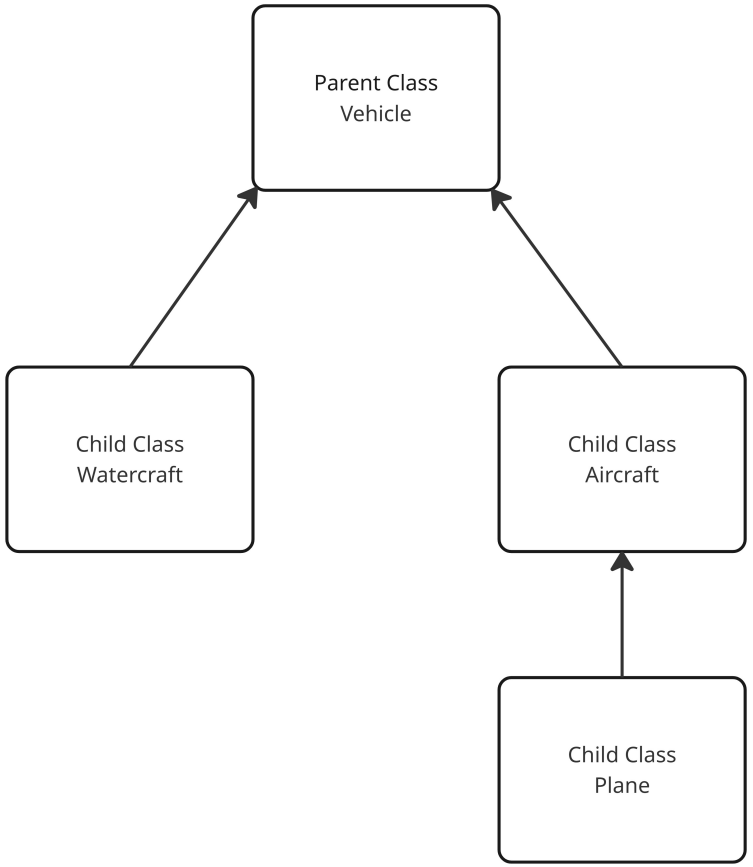
Solution: **inheritance**

- Create a base class
- Only modify the nuances
- Properties can be "inherited"

Inheritance

- **Parent class** / **superclass**: more general
- **Child class** / **subclass**: class that inherits from a super class
- "**is a**" relation

- Inheritance is **transitive**
  → Plane **is a** vehicle



Inheritance

Implementation

- Keyword: **`extends`**
- Transitivity is implicit: Plane also is a Vehicle
  $\rightarrow$ no need to write
  `Plane extends Aircraft extends..`
- In java class can only extend **one** class

```java
public class Vehicle {...}

public class Watercraft extends Vehicle {...}

public class Aircraft extends Vehicle {...}

public class Plane extends Aircraft {...}

// Not possible:
public class Plane extends Aircraft, Watercraft {...}

```

Inheritance

Implementation

```java
public class Aircraft {
    String aircraftId;

    void fly() {
        // ...
    }
}

public class Plane extends Aircraft {
    Flaps flaps;

    void controlFlaps() {
        // ...
    }

    public static void main(String[] args) {
        Plane plane = new Plane();

        plane.fly();  // Inherited
        plane.aircraftId;  // Inherited
        plane.controlFlaps();  // Own method
    }
}

```

It will inherit:

- Attributes
- Methods
- Parent classes
- **No** constructors!!

But subclasses can

- Add new methods, attributes
- Redefine (overwrite) the defined methods, attributes

Overwriting

Methods

- Different classes must have different method implementations
- Same signatures
  - However the **return type** can be **more specific**
  - e.g. `Integer` subclass of `Number`
    → subclass uses `Integer` instead
- Annotation: `@Override`

```java
public class LandVehicle {
    public void move() {
        // ...
    }

    public Number calculateFuel() {
        // ...
    }
}

public class Car extends LandVehicle {
    @Override
    public void move() {
        // Overwrites the method
    }

    @Override
    public Integer calculateFuel() {
        // Also uses more specific return type
    }
}
```

Overwriting

Attributes

- Attributes **won't** be **overwritten**, instead redeclared/shadowed!
- Usually not very healthy to have attributes to be overwritten in the base class → don't do it

```java
public class Vehicle {
    Date buildDate;
    int tireAmounts = 1;
}

public class Car extends Vehicle {
    // Shadowed:
    int tireAmounts = 4;

    public static vodi main(String[] args) {
        Car car = new Car();
        int amount = car.tireAmounts;  // = 4
    }
}

```

Superclass

- Can also reach/use the methods and attributes of the superclasses
- Keyword: **super**

- Subclasses can still implement new properties but the superclass properties still available

```java
public class Vehicle {
    int wheels = 4;

    public void start() {...}
}

public class Car extends Vehicle {
    int wheels = 6;

    @Override
    public void start() {...}

    public void demo() {
        this.start();   // calls Car.start()
        super.start();  // calls Vehicle.start()
        this.wheels;    // = 4
        super.wheels;   // = 6
    }
}
```

this: Refers to the current object

super: Refers to the same object, but you explicitly access the namespace of the superclass

Constructors

- Constructors won't be inherited!
- Can call the superclass constructor with **super**
  - Java calls `super()` implicitly
  - If superclass doesn't have the default constructor → error

```java
public class Animal {
    int age;

    public Animal(int age) {
        this.age = age;
    }
}

public class Human extends Animal {
    String name;

    public Human(int age, String name) {
        super(age);  // <-- Necessary
        this.name = name;
    }
}

```

Object Class

- Every class is a subclass of `Object`.
- Important methods:
  - `toString()`
  - `equals(Object obj)`
  - `hashCode()`
  - `getClass()`
  - `clone()`
- These methods can be overwritten if necessary

```java
Human human = new Human();
Object object = (Object) human;  // Always possible

```

Important

Never use `Objects` as parameters or elsewhere!
Use the specified classes instead!
Except for `equals(Object obj)`

Guideline: https://sdq.kastel.kit.edu/programmieren/

Final Modifier

- Already seen: `final` attributes can't be modified

- Classes can also be `final` → prevents inheritance
- Methods can be `final` → prevents overwriting

```
public final class Utility {...}

public class UtilityExtension extends Utility {
    // ERROR
}

```

Dynamic Binding

- Static data type → type in source code
- Dynamic data type → real type of the object
- Which method will be used is decided at runtime → the dynamic type

```java
public class Animal {
    public void speak() {
        System.out.println("Animal speaks");
    }
}

public class Dog extends Animal {
    @Override
    public void speak() {
        System.out.println("Dog barks");
    }
}

```

```java
public static void main(String[] args) {
    Animal foo = new Dog();
    foo.speak();  // output: "Dog barks"
}

```

- `Animal` is the **static type** for `foo`
- `Dog` is the **dynamic type** for `foo`
  → hence `Dog.speak` is called

Dynamic Binding

Example

```java
Animal firstAnimal = new Dog();
firstAnimal.makeSound();

// Output: "woof"

Labrador labrador = new Labrador();
labrador.makeSound();

// Output: "hi there"

Animal secondAnimal = (Animal) labrador;
secondAnimal.makeSound();

// Output: "hi there"

```

```java
public class Animal {
    public void makeSound() {
        System.out.println("");
    }
}

public class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("woof");
    }
}

public class Labrador extends Dog {
    @Override
    public void makeSound() {
        System.out.printl("hi there");
    }
}

```

Dynamic Binding

Example

```java
Labrador labrador = new Labrador();
labrador.makeSound();

```

Outputs:

1. "Hi there"
2. "woof"
3. "Hi there"

```java
public class Dog extends Animal {
    public void bark() {
        System.out.println("woof");
    }

    @Override
    public void makeSound() {
        bark();
    }
}

public class Labrador extends Dog {
    @Override
    public void bark() {
        System.out.printl("hi there");
    }

    @Override
    public void makeSound() {
        bark();
        super.bark();
        super.makeSound();
    }
}

```

Exercise 1

Präsenzübung WS 23/24

```java
class Parent {
    public String getFoo() { return "parentFoo"; }
}
class Child extends Parent {
    public String getFoo() { return "childFoo"; }
    public String getBar() { return "childBar"; }
}
class GrandChild extends Child {
    public String getFoo() { return "grandChildFoo"; }
}
Parent p0 = new Child();
p0.getFoo();  // Returns: ???
p0.getBar();

Child c0 = new GrandChild();
c0.getFoo();
c0.getBar();

GrandChild gc0 = new GrandChild();
gc0.getFoo();
gc0.getBar();

```

- What are the returned values?

Abstract Class

Problem

- Every animal make different sounds
- How to specify a common sound?

- More general: how can one model a generic method?
- Why even define the method if it's going to be overwritten each time?
- If we don't define such method then how do we enforce that all animals make sounds?

```java
public class Animal {
    public void makeSound() {
        // What to do here?
    }
}

public class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Woof");
    }
}

```

Abstract Class

Idea

```java
public abstract class Animal {
    public abstract void makeSound();

    public void sleep() {
        System.out.println("ZzzZzzZzz");
    }
}

public class Dog extends Animal {
    @Override  // <- Has to define the method
    public void makeSound() {
        System.out.println("Woof");
    }

    public static void main(String[] args) {
        Animal dog = new Dog();
        dog.makeSound();
        dog.sleep();

        Animal animal = new Animal();  // Not possible -> error
    }
}

```

- We create **abstract** classes
  $\rightarrow$ these classes can't be initialized
- We specify method signatures and don't define them
- The implementation then can change between classes

Command Pattern

Idea

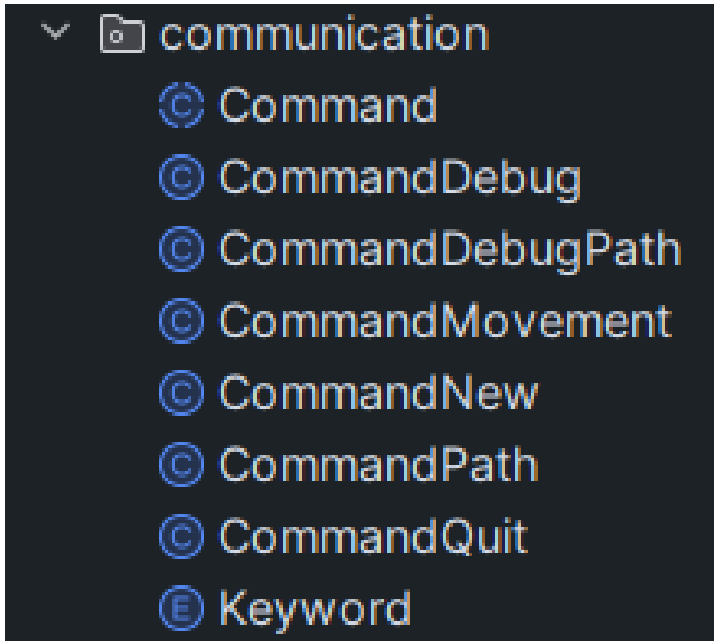
Separate the program logic of different commands

- Commands do not directly influence each other
- Patterns are easy to understand
- Patterns can be reused
- Helps enormously starting from sheet 4 :)

Command Pattern

Implementation

- Have a `Command` abstract class
- Create a new class for each command (e.g. `CommandQuit`)
- Have a `Keyword` enum for the keywords of each command



Command Pattern

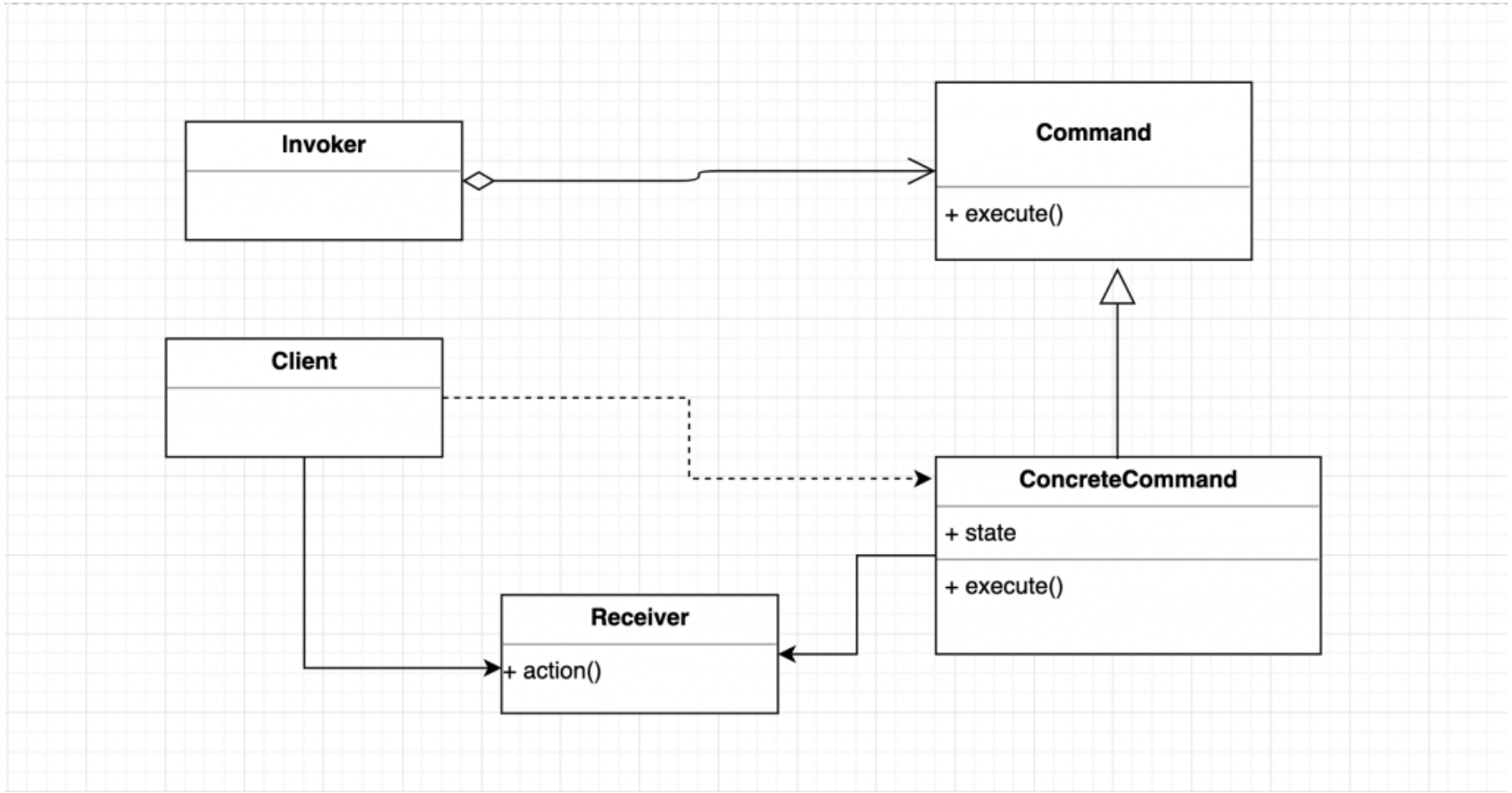
Implementation

- `Commands` will have an `execute` method
- This method will require instances of other classes
- This method will execute the necessary action when called

```java
@Override
public boolean execute(Navigation navigation, Serializer serializer) {
    List<Vector2D> coordinates = navigation.findPath().getCoordinates();

    if (coordinates.isEmpty()) {
        System.out.println(CommandPath.MESSAGE_NO_PATH);
        return true;
    }

    String map = serializer.serialize(...);
    System.out.println(map);
    return true;
}

```

Command Pattern

Formal



Exercise 2

```java
public abstract class Student {
    private final String name;
    private final int id;

    public Student(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public abstract String getMajor();
    public abstract List<Module> getRequiredModules();

    public float getRequiredCredits() {
        // TODO: Implement me
    }

    @Override
    public String toString() {
        return id + ": " + name + " (" + getMajor() + ")";
    }
}

```

- Model a `CsStudent` class that extends `Student`
- `getRequiredModules()` should return the following for a `CsStudent`: `ModuleProgramming`, `ModuleAlgebra`, `ModuleGBI`
- Each module extends the following class:

```java
pbulic abstract class Module {
    public abstract boolean passed();
    public abstract float getCredits();
}

```

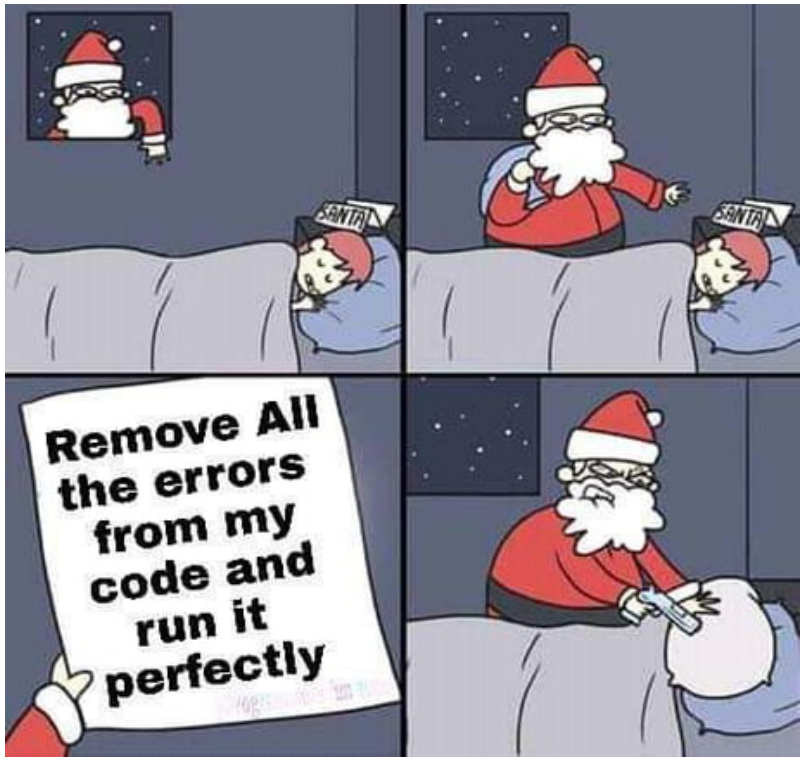
- `getRequiredCredits()` should return the sum of credits gained from the modules of student

Conclusion

- Arrays
- Multidimensional arrays
- Encapsulation
- Access modifiers
- Type conversions
- `toString() & equals()`

Reminder

No tutorial next week!!



Exercise 1

```java
class Parent {
    public String getFoo() { return "parentFoo"; }
}
class Child extends Parent {
    public String getFoo() { return "childFoo"; }
    public String getBar() { return "childBar"; }
}
class GrandChild extends Child {
    public String getFoo() { return "grandChildFoo"; }
}
Parent p0 = new Child();
p0.getFoo();  // childFoo
p0.getBar();  // Error

Child c0 = new GrandChild();
c0.getFoo();  // grandChildFoo
c0.getBar();  // childBar

GrandChild gc0 = new GrandChild();
gc0.getFoo();  // grandChildFoo
gc0.getBar();  // childBar

```

Exercise 2

```java
class CsStudent extends Student {
    private static final String MAJOR = "Computer Science";
    private List<Module> modules;

    public CsStudent(String name, int id) {
        super(name, id);
        this.modules = List.of(
            new ModuleProgramming(),
            new ModuleAlgebra(),
            new ModuleGBI()
        );
    }

    @Override
    public String getMajor() {
        return MAJOR;
    }

    @Override
    public List<Module> getRequiredModules() {
        return List.copyOf(modules);
    }
}

```

Exercise 2

```java
abstract class Student {
    private final String name;
    private final int id;

    public Student(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public abstract String getMajor();
    public abstract List<Module> getRequiredModules();

    public float getRequiredCredits() {
        float sum = 0;
        for (Module module : getRequiredModules()) {
            sum += module.getCredits();
        }
        return sum;
    }

    @Override
    public String toString() { return id + ": " + name + " (" + getMajor() + ")"; }
}

```